

Curso de Python Básico: "Primeiros Passos com Python" 🐍

Este curso irá guiá-lo através dos fundamentos da programação em Python, capacitando-o a escrever seus primeiros scripts.

Módulo 1: Introdução ao Python

- **O que é Python?**
 - Uma linguagem de programação de alto nível, interpretada, interativa e orientada a objetos.
 - Conhecida por sua sintaxe clara e legibilidade.
- **Por que aprender Python?**
 - Versátil: desenvolvimento web, análise de dados, inteligência artificial, automação, etc.
 - Grande comunidade e muitas bibliotecas.
 - Fácil de aprender para iniciantes.
- **Instalação do Python:**
 - Acesse python.org e baixe a versão mais recente.
 - Siga as instruções de instalação para o seu sistema operacional (Windows, macOS, Linux).
 - Certifique-se de marcar a opção "Add Python to PATH" durante a instalação no Windows.
- **Primeiro Contato: O Interpretador Python**
 - Abra o terminal ou prompt de comando.
 - Digite `python` (ou `python3` em alguns sistemas) e pressione Enter.
 - Você verá o prompt `>>>`. Experimente digitar `print("Olá, Mundo!")` e pressione Enter.

Módulo 2: Sintaxe Básica e Tipos de Dados

- **Variáveis:**
 - Nomes que armazenam dados.
 - Exemplo: `nome = "Maria", idade = 30, altura = 1.75`
 - Python é dinamicamente tipado (você não precisa declarar o tipo da variável).
- **Comentários:**
 - Linhas no código que não são executadas.
 - Usados para explicar o código.
 - Começam com `#`. Exemplo: `# Isto é um comentário`
- **Tipos de Dados Fundamentais:**
 - **Inteiros (int)**: Números inteiros. Ex: `10, -5, 0`
 - **Ponto Flutuante (float)**: Números com casas decimais. Ex: `3.14, -0.5, 2.0`
 - **Strings (str)**: Sequências de caracteres (texto). Envolvidas por aspas simples (`'`) ou duplas (`"`). Ex: `"Olá", 'Python'`
 - **Booleanos (bool)**: Representam verdadeiro (`True`) ou falso (`False`).
- **Função print():**
 - Usada para exibir valores na tela.
 - Exemplo: `print(nome), print("Sua idade é:", idade)`
- **Função input():**
 - Usada para receber dados do usuário.
 - Sempre retorna uma string.
 - Exemplo: `nome_usuario = input("Digite seu nome: ")`
- **Conversão de Tipos (Type Casting):**
 - `int()`: Converte para inteiro.
 - `float()`: Converte para ponto flutuante.
 - `str()`: Converte para string.
 - Exemplo: `numero_texto = "10", numero_inteiro = int(numero_texto)`

```
# Exemplo Módulo 2
nome_aluno = input("Digite o nome do aluno: ")
nota1_str = input("Digite a primeira nota: ")
nota2_str = input("Digite a segunda nota: ")

# Convertendo as notas para float
nota1 = float(nota1_str)
nota2 = float(nota2_str)

media = (nota1 + nota2) / 2

print("Aluno:", nome_aluno)
print("Média:", media)
print(type(media)) # Mostra o tipo da variável media
```

Módulo 3: Operadores

- **Operadores Aritméticos:**
 - `+` (Adição)

- - (Subtração)
- * (Multiplicação)
- / (Divisão - resulta em float)
- // (Divisão inteira - descarta a parte decimal)
- % (Módulo - resto da divisão)
- ** (Exponenciação)
- **Operadores de Comparação (resultam em Booleanos):**
 - == (Igual a)
 - != (Diferente de)
 - > (Maior que)
 - < (Menor que)
 - >= (Maior ou igual a)
 - <= (Menor ou igual a)
- **Operadores Lógicos:**
 - and (E lógico - verdadeiro se ambas as condições forem verdadeiras)
 - or (OU lógico - verdadeiro se pelo menos uma condição for verdadeira)
 - not (NÃO lógico - inverte o valor booleano)

```
# Exemplo Módulo 3

a = 10
b = 3

soma = a + b
print("Soma:", soma) # Saída: Soma: 13

divisao_inteira = a // b
print("Divisão Inteira:", divisao_inteira) # Saída: Divisão Inteira: 3

resto = a % b
print("Resto:", resto) # Saída: Resto: 1

print("a é maior que b?", a > b) # Saída: a é maior que b? True

idade_pessoa = 20
tem_carteira = True
pode_dirigir = idade_pessoa >= 18 and tem_carteira
print("Pode dirigir?", pode_dirigir) # Saída: Pode dirigir? True
```

Módulo 4: Estruturas de Controle de Fluxo

- **Estruturas Condicionais:**
 - **if**: Executa um bloco de código se uma condição for verdadeira.
 - **elif (else if)**: Verifica outra condição se a(s) anterior(es) for(em) falsa(s).
 - **else**: Executa um bloco de código se todas as condições anteriores forem falsas.
 - **Indentação é crucial em Python!** O código dentro de `if`, `elif` e `else` deve ser indentado (geralmente 4 espaços).

```
# Exemplo if-elif-else
temperatura = 25

if temperatura > 30:
    print("Está muito quente! ☹️")
elif temperatura > 20:
    print("Clima agradável. 😊")
else:
    print("Está frio! ❄️")
```

- **Estruturas de Repetição (Loops):**
 - **for loop**: Usado para iterar sobre uma sequência (como uma string, lista, ou `range()`).
 - `range(n)`: Gera uma sequência de números de 0 até n-1.
 - `range(inicio, fim)`: Gera números de inicio até fim-1.
 - `range(inicio, fim, passo)`: Gera números de inicio até fim-1, com um passo.

```
# Exemplo for loop
print("Contando até 4:")
for i in range(5): # de 0 a 4
    print(i)

print("\nFrutas:")
frutas = ["maçã", "banana", "laranja"]
for fruta in frutas:
    print(fruta)
```

```
* **`while` loop:** Executa um bloco de código enquanto uma condição for verdadeira.
* Cuidado com loops infinitos! Certifique-se de que a condição eventualmente se torne falsa.
```

```
# Exemplo while loop
contador = 0
while contador < 5:
    print("Contador:", contador)
    contador = contador + 1 # ou contador += 1
```

```
* **Controle de Loops:**
* `break`: Interrompe o loop imediatamente.
* `continue`: Pula para a próxima iteração do loop.
```

Módulo 5: Estruturas de Dados Básicas

- **Listas (list):**
 - Coleções ordenadas e mutáveis de itens.
 - Podem conter itens de tipos diferentes.
 - Definidas com colchetes `[]`.
 - Acesso a elementos por índice (começando em 0).
 - Métodos úteis: `append()`, `insert()`, `remove()`, `pop()`, `len()`.

```
# Exemplo Listas
numeros = [1, 2, 3, 4, 5]
nomes = ["Ana", "Carlos", "Beatriz"]

print("Primeiro número:", numeros[0]) # Saída: Primeiro número: 1
nomes.append("Daniel")
print("Nomes atualizados:", nomes) # Saída: Nomes atualizados: ['Ana', 'Carlos', 'Beatriz', 'Daniel']
print("Tamanho da lista de números:", len(numeros)) # Saída: Tamanho da lista de números: 5
```

- **Tuplas (tuple):**
 - Coleções ordenadas e **imutáveis** de itens.
 - Uma vez criadas, não podem ser alteradas.
 - Definidas com parênteses `()`.
 - Úteis para dados que não devem mudar.

```
# Exemplo Tuplas
coordenadas = (10.5, -8.2)
cores_rgb = (255, 0, 128)

print("Latitude:", coordenadas[0]) # Saída: Latitude: 10.5
# coordenadas[0] = 5.0 # Isto geraria um erro (TypeError)
print("Número de cores:", len(cores_rgb)) # Saída: Número de cores: 3
```

Módulo 6: Funções

- **O que são Funções?**
 - Blocos de código reutilizáveis que realizam uma tarefa específica.
 - Ajudam a organizar o código e torná-lo mais legível.
- **Definindo uma Função:**

- Usa a palavra-chave `def`, seguida pelo nome da função, parênteses `()` e dois pontos `:`.
- O corpo da função é indentado.
- Pode receber **parâmetros** (argumentos) dentro dos parênteses.
- Pode **retornar** um valor usando a palavra-chave `return`.

```
# Exemplo definindo e chamando uma função
def saudacao(nome):
    """Esta função cumprimenta a pessoa passada como parâmetro.""" # Docstring
    print(f"Olá, {nome}! Bem-vindo(a).")

def somar(a, b):
    """Esta função retorna a soma de dois números."""
    resultado = a + b
    return resultado

# Chamando as funções
saudacao("Alice") # Saída: Olá, Alice! Bem-vindo(a).
saudacao("Bruno") # Saída: Olá, Bruno! Bem-vindo(a).

valor_soma = somar(5, 7)
print("A soma é:", valor_soma) # Saída: A soma é: 12
print("3 + 8 é:", somar(3, 8)) # Saída: 3 + 8 é: 11
```

- **Escopo de Variáveis:**
 - Variáveis definidas dentro de uma função são **locais** (só existem dentro da função).
 - Variáveis definidas fora de todas as funções são **globais**.

Módulo 7: Pequeno Projeto Prático 📁

Vamos criar um jogo simples de "Adivinhe o Número"!

Objetivo: O computador escolhe um número aleatório e o jogador tenta adivinhá-lo.

```
import random # Módulo para gerar números aleatórios

def jogo_adivinhe_o_numero():
    numero_secreto = random.randint(1, 20) # Gera um número aleatório entre 1 e 20
    tentativas = 0
    max_tentativas = 5

    print("Bem-vindo ao 'Adivinhe o Número'!")
    print(f"Eu pensei em um número entre 1 e 20. Você tem {max_tentativas} tentativas.")

    while tentativas < max_tentativas:
        try:
            palpite_str = input(f"Tentativa {tentativas + 1}/{max_tentativas}. Qual o seu palpite? ")
            palpite = int(palpite_str)
        except ValueError:
            print("Por favor, digite um número válido.")
            continue # Volta para o início do loop

        tentativas += 1

        if palpite < numero_secreto:
            print("Muito baixo!")
        elif palpite > numero_secreto:
            print("Muito alto!")
        else:
            print(f"Parabéns! Você acertou o número {numero_secreto} em {tentativas} tentativas! 🎉")
            return # Termina a função (e o jogo)

    print(f"Fim de jogo! Você não conseguiu adivinhar. O número era {numero_secreto}. 😊")

# Iniciar o jogo
jogo_adivinhe_o_numero()
```

Como rodar o projeto:

1. Salve o código acima em um arquivo chamado `jogo.py`.
 2. Abra o terminal ou prompt de comando.
 3. Navegue até a pasta onde você salvou o arquivo.
 4. Execute o comando: `python jogo.py`
-

Próximos Passos 📖

- **Pratique, pratique, pratique!** Crie pequenos programas e desafios para si mesmo.
- Explore mais sobre **strings** e seus métodos.
- Aprenda sobre **dicionários (dict)** e **conjuntos (set)**.
- Entenda o conceito de **Orientação a Objetos (OOP)** em Python (classes e objetos).
- Trabalhe com **arquivos** (leitura e escrita).
- Explore **bibliotecas Python** populares como `NumPy`, `Pandas` (para dados), `Matplotlib` (para gráficos), `Requests` (para web), `Flask / Django` (para desenvolvimento web).

Espero que este curso básico ajude você a começar sua jornada com Python! Boa codificação! 😊